

Informe final, Desafío#1

Oscar Mauricio Echeverri Parra.

Luna Jaramillo Guzmán.

En este proyecto, creamos una adaptación del juego Sweet Crush utilizando C++. La meta fundamental fue representar cada ficha del tablero empleando solo 3 bits por posición, en lugar de recurrir a un tipo de dato completo para cada una.

A lo largo del desarrollo, nos enfocamos en operaciones de bits, gestión de memoria dinámica, punteros, arreglos y la organización del código en distintos archivos. También implementamos las funciones esenciales para visualizar el tablero, eliminar fichas, identificar combinaciones, aplicar la gravedad, generar fichas nuevas y ajustar las dimensiones del tablero.

El desafío principal del proyecto fue optimizar el almacenamiento de múltiples fichas utilizando la menor cantidad de memoria posible. Cada posición del tablero ocupa exactamente 3 bits, lo que permite representar hasta 8 valores diferentes.

Las letras de la A a la F simbolizan las seis fichas empleadas en el juego. La letra V se usa para indicar que una posición está vacía, mientras que la R se aplica para mostrar temporalmente una posición reservada durante el transcurso de una jugada.

Organización de la memoria

Para calcular la memoria necesaria primero obtenemos el número de posiciones:

Posiciones = filas $\times$ columnas

Después:

Bits = posiciones $\times$ 3

Y finalmente calculamos cuántos bytes son necesarios, teniendo en cuenta que un byte contiene 8 bits.

Por ejemplo, para un tablero de 2×3 :

Posiciones = 2×3

Posiciones = 6

Bits = 6×3

Bits = 18

Bytes = $18 / 8$

Bytes = 2,25

Como no podemos reservar 2,25 bytes, se necesitan: 3 bytes

Esto se realiza en la función `calcularBytes()`.

Uso de memoria dinamica.

Para almacenar el tablero, empleamos: `unsigned char* tablero;` .

Esto nos permite que el tamaño de la memoria se ajuste a las dimensiones actuales del tablero.

La memoria se asigna dinámicamente usando:

`new unsigned char[bytes];` ,

y se libera después con: `delete[] tablero;` .

De este modo, evitamos utilizar una matriz fija de tamaño máximo, permitiendo que la memoria se adecue al tamaño del tablero.

Acceso a una ficha mediante bits.

Para localizar una ficha, se utiliza su posición en el tablero.

Primero se calcula:

```
int bitInicio = posicion * 3;
```

Esto determina en qué bit comienza la ficha.

Luego se calcula:

```
int numeroByte = bitInicio / 8;
```

para identificar en qué byte inicia.

También se calcula:

```
int desplazamiento = bitInicio % 8;
```

para determinar cuántos bits se deben desplazar dentro de ese byte.

La función que lleva a cabo este proceso se llama `obtenerFicha`.

Esta función emplea desplazamientos y máscaras para extraer únicamente los 3 bits correspondientes a la ficha.

fichas que quedan divididas en dos bytes

Uno de los desafíos más significativos fue la discrepancia entre los 3 bits que usa una ficha y los 8 bits que contiene un byte. Esto significa que no todas las fichas pueden almacenarse completamente dentro de un solo byte.

Cuando los 3 bits caben en el mismo byte, se puede usar directamente una máscara:

```
(tablero[numeroByte] >> desplazamiento) & 7.
```

Sin embargo, si la ficha se divide entre dos bytes, el programa debe tomar una parte del primer byte y otra del siguiente. La función `obtenerFicha()` está diseñada para manejar ambos escenarios, lo que permite acceder de manera correcta a cualquier ficha, incluso si está situada justo en el límite entre bytes.

Modificación de una ficha.

Para modificar una posición, empleamos la función `ponerFicha`. Esta función también calcula `bitInicio`, `numeroByte` y `desplazamiento`. Luego, utiliza operaciones para borrar los bits anteriores y colocar los nuevos 3 bits. Esto permite actualizar una ficha sin necesidad de transformar todo el tablero a una estructura diferente.

visualización tablero.

Aunque internamente las fichas se almacenan como números de 3 bits, al mostrar el tablero se convierten a letras.

1 → A

2 → B

3 → C 0 → V

4 → D 7 → R

5 → E

6 → F

```
JuegoSweetCrush - Debug
=====
MENU
=====
1. Mostrar tablero
2. Realizar jugada
3. Mostrar bits
4. Mostrar Fichas en binario
5. Agregar fila
6. Eliminar fila
7. Agregar columna
8. Eliminar columna
9. Mostrar estadísticas
0. Salir
Seleccione una opción: 1

Tablero:
D D B B F D
D D D F F F
D A B C E E
D B F D E C
C B D B E B
B D E E C B
```

Tablero:

D	D	B	B	F	D
D	D	D	F	F	F
D	A	B	C	E	E
D	B	F	D	E	C
C	B	D	B	E	B
B	D	E	E	C	B

cuando se realiza una jugada

Cuando el usuario selecciona la opción:

2. Realizar jugada

el programa solicita la fila y la columna.

La función `pedirPosicion()` verifica que la posición esté dentro de los límites del tablero.

Después se obtiene la ficha mediante: `obtenerFicha()`

Si la posición está vacía, se informa al usuario.

Si contiene una ficha, primero se muestra como reservada: R

y posteriormente se coloca como: V para representar que fue eliminada

```
Ingrese la fila: 1
Ingrese la columna: 2
```

```
Ficha seleccionada (R = Reservada):
```

```
E E E
B C R
A F B
C D F
```

```
Ficha eliminada (V = Vacio):
```

```
E E E
B C V
A F B
C D F
```

```
Jugada realizada.
```

```
Cascadas generadas: 1
```

```
Tablero despues de la jugada:
```

```
E E F
B C E
A F B
C D F
```

Eliminación de combinaciones

La función: `detectarCombinaciones()`

recorre el tablero buscando fichas iguales consecutivas.

Primero revisa las combinaciones horizontales.

Por ejemplo:

ABBB

CDEF

ACDE

Ficha eliminada (V = Vacío):

```
V B C
B C D
C D E
F F F
```

Jugada realizada.

Cascadas generadas: 1

Tablero despues de la jugada:

```
B C D
E B C
B C D
C D E
```

Después también revisa las combinaciones verticales.

Por ejemplo:

ABC

ADC

AEC

Ficha eliminada (V = Vacío):

```
D E V
D D C
E D E
A D F
```

Jugada realizada.

Cascadas generadas: 1

Tablero despues de la jugada:

```
D E D
D D C
E C E
A E F
```

Cuando encuentra tres o más fichas iguales, las posiciones se marcan en el arreglo: `bool* eliminados`

Esto permite marcar todas las posiciones antes de eliminarlas.

Eliminación de fichas

Después de detectar las combinaciones se utiliza:

`eliminarCombinaciones`

Esta función recorre las posiciones marcadas y las convierte en: 0

El 0 representa: V = Vacío

Ficha eliminada (V = Vacío):

```
E E E
B C V
A F B
C D F
```

→ 0

Aplicación de gravedad

Después de eliminar las fichas quedan espacios vacíos.

La función: `aplicarGravedad`

recorre cada columna desde abajo y mueve las fichas hacia las posiciones inferiores.

Por ejemplo:

A

V

B

V

puede reorganizarse como:

V

V

A

B

De esta manera las fichas ocupan los espacios disponibles desde abajo.

Generación de nuevas fichas.

Después de aplicar gravedad pueden quedar espacios vacíos en la parte superior.

La función: `generarNuevas Fichas`

busca las posiciones que tienen valor 0 y genera una nueva ficha entre A y F. Internamente se utiliza: $\text{rand\% } 6 + 1$

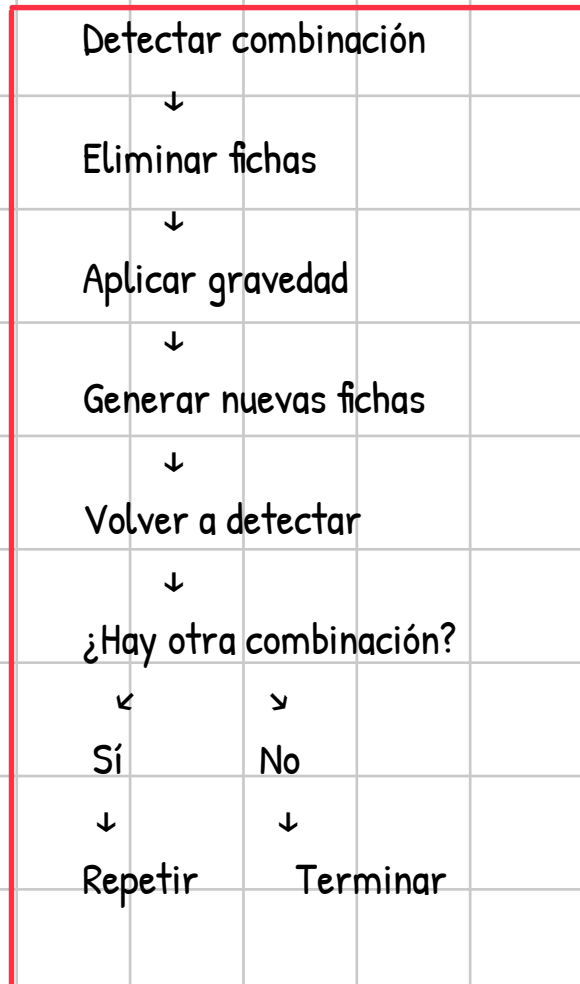
porque los valores del 1 al 6 corresponden a las seis fichas.

Cascadas

Después de generar nuevas fichas, el programa vuelve a revisar si existen nuevas combinaciones.

Esto se realiza mediante: procesar Cascadas

El proceso es:



Cada vez que se completa este proceso se aumenta el número de cascadas.

También se actualizan:

fichas eliminadas:

combinaciones

puntaje

cascadas.

Puntaje.

El programa asigna: $\text{puntaje} += \text{cantidadEliminados} * 10;$

Por lo tanto, por cada ficha eliminada se suman 10 puntos.

Por ejemplo, si en una combinación se eliminan 3 fichas:

$$3 \times 10 = 30 \text{ puntos}$$

Agregar eliminar filas

El programa permite modificar las dimensiones del tablero.

Para agregar una fila se utiliza: `agregarFila`

Se crea un nuevo tablero con una fila adicional y se copian las fichas existentes.

La nueva fila recibe fichas aleatorias.

Además, la nueva fila puede colocarse en diferentes posiciones, no solamente al final.

Para eliminar una fila se utiliza: `eliminarFila`

Se crea un nuevo tablero con una fila menos y se conservan las demás fichas.

Agregar eliminar columnas

El mismo procedimiento se utiliza para las columnas.

Para agregar una columna:

En ambos casos, se genera temporalmente un nuevo tablero con las dimensiones adecuadas, donde se copian las fichas que deben mantenerse. Esto garantiza que se preserven las fichas necesarias mientras se realiza la modificación.

Manejo de memoria al cambiar el tablero

Un aspecto fundamental fue gestionar la cantidad de memoria utilizada al modificar las dimensiones del tablero. La función `guardarNuevoTablero` calcula los bytes necesarios para el nuevo tablero. Cuando se agrega una fila o columna y se requiere más memoria, se libera la memoria anterior y se reserva una nueva.

En el caso de que se elimine una fila o columna, el programa evalúa el porcentaje de utilización de la memoria. En el código, se establece un límite del 65%. Si la utilización es inferior a ese porcentaje, se ajusta la memoria para adaptarla al nuevo tamaño.

Estadísticas

El programa tiene una opción para mostrar las estadísticas actuales.

Se muestran:

```
=====
ESTADISTICAS
=====
Filas actuales: 4
Columnas actuales: 3
Bytes asignados: 5
Eliminaciones del usuario: 1
Fichas eliminadas: 3
Combinaciones detectadas: 1
Cascadas: 1
Puntaje: 30
```

Problemas encontrados en el desarrollo

Uno de los retos principales fue entender cómo almacenar una ficha de 3 bits, especialmente cuando podía dividirse entre dos bytes. Además, se debía tener cuidado con las posiciones, ya que el tablero se gestiona de manera lineal utilizando la fórmula:

$$\text{posicion} = \text{fila} * \text{columnas} + \text{columna}.$$

Otro factor crítico fue modificar filas y columnas sin perder las fichas ya almacenadas. También se requirió un control adecuado de la memoria dinámica para evitar el uso excesivo de memoria, asegurando que se mantuviera únicamente la cantidad necesaria. Todo esto fue fundamental para el correcto funcionamiento del sistema.

Pruebas

Durante las pruebas se verificaron diferentes partes del programa mediante el menú de la consola.

Se pudo observar el tablero utilizando las letras A-F, además de las representaciones v y R.

También se utilizaron las opciones:

```
=====
                        MENU
=====
1. Mostrar tablero
2. Realizar jugada
3. Mostrar bits
4. Mostrar Fichas en binario
5. Agregar fila
6. Eliminar fila
7. Agregar columna
8. Eliminar columna
9. Mostrar estadísticas
0. Salir
Seleccione una opción: █
```

Para las pruebas de las jugadas, se seleccionaron posiciones utilizando la fila y la columna, lo que permitió observar cómo se comportaba el tablero tras la eliminación de una ficha. Este enfoque facilitó la verificación de que las fichas se gestionaban correctamente y que no había pérdida de información ni errores en la lógica del almacenamiento. Se analizó cómo las operaciones de eliminación afectaban la estructura del tablero y la integridad de los datos almacenados.

vamos a jugar.

Generación tablero: 4X3

```
Terminal
JuegoSweetCrush - Debug x
=====
SWEET CRUSH
=====
Ingrese el numero de filas: 4
Ingrese el numero de columnas: 3

=====
MENU
=====
1. Mostrar tablero
2. Realizar jugada
3. Mostrar bits
4. Mostrar Fichas en binario
5. Agregar fila
6. Eliminar fila
7. Agregar columna
8. Eliminar columna
9. Mostrar estadísticas
0. Salir
Seleccione una opcion: 1

Tablero:
A B C
D E F
D R V
D A V
```

Realizar jugada.

```
JuegoSweetCrush - Debug
0. Salir
Seleccione una opcion: 2

Seleccione la ficha que desea eliminar.
Ingrese la fila: 0
Ingrese la columna: 0

Ficha seleccionada (R = Reservada):
R B C
D E F
D R V
D A V

Ficha eliminada (V = Vacio):
V B C
D E F
D R V
D A V

Jugada realizada.
Cascadas generadas: 1

▲ Tablero despues de la jugada:
C B C
C E E
E R C
E A F
```

Despues de la jugada verificamos las estadisticas.

```
JuegoSweetCrush - Debug
5. Agregar fila
6. Eliminar fila
7. Agregar columna
8. Eliminar columna
9. Mostrar estadisticas
0. Salir
Seleccione una opcion: 9

=====
ESTADISTICAS
=====
Filas actuales: 4
Columnas actuales: 3
Bytes asignados: 5
Eliminaciones del usuario: 1
Fichas eliminadas: 3
Combinaciones detectadas: 1
Cascadas: 1
Puntaje: 30
```

Aquí podemos verificar que obtuvimos 1 Cascada se actualizo el puntaje y los demás items

Mostrar bits.

```
=====
MENU
=====
1. Mostrar tablero
2. Realizar jugada
3. Mostrar bits
4. Mostrar Fichas en binario
5. Agregar fila
6. Eliminar fila
7. Agregar columna
8. Eliminar columna
9. Mostrar estadísticas
0. Salir
Seleccione una opción: 3

Bits del tablero:
11001011 01101011 01101111 11010110 00110000
```

Mostrar fichas en binario

```
JuegoSweetCrush - Debug [X]

MENU
=====
1. Mostrar tablero
2. Realizar jugada
3. Mostrar bits
4. Mostrar Fichas en binario
5. Agregar fila
6. Eliminar fila
7. Agregar columna
8. Eliminar columna
9. Mostrar estadísticas
0. Salir
Seleccione una opción: 4

Fichas en binario:
011 010 011
011 101 101
101 111 011
101 001 110
```

Aquí podemos verificar
que las fichas se están
representando correctamente

Agregar fila.

```
JuegoSweetCrush - Debug [X]

=====
MENU
=====
1. Mostrar tablero
2. Realizar jugada
3. Mostrar bits
4. Mostrar Fichas en binario
5. Agregar fila
6. Eliminar fila
7. Agregar columna
8. Eliminar columna
9. Mostrar estadísticas
0. Salir
Seleccione una opcion: 5

Ingrese la posicion de la nueva fila: 3
Fila agregada correctamente.
```

```
=====
1. Mostrar tablero
2. Realizar jugada
3. Mostrar bits
4. Mostrar Fichas en binario
5. Agregar fila
6. Eliminar fila
7. Agregar columna
8. Eliminar columna
9. Mostrar estadísticas
0. Salir
Seleccione una opcion: 1

Tablero:
C B C
C E E
E R C
C E A
E A F
```

Aquí verificamos que se agregó correctamente la fila donde se le indicó. De modo que las dimensiones del tablero también cambiarán

Realizar jugada.

```
JuegoSweetCrush - Debug [X]

7. Agregar columna
8. Eliminar columna
9. Mostrar estadísticas
0. Salir
Seleccione una opcion: 2

Seleccione la ficha que desea eliminar.
Ingrese la fila: 2
Ingrese la columna: 0

Ficha seleccionada (R = Reservada):
C B C
C E E
R R C
C E A
E A F

Ficha eliminada (V = Vacio):
C B C
C E E
V R C
C E A
E A F

Jugada realizada.
Cascadas generadas: 1

Tablero despues de la jugada:
A B C
D E E
D R C
B E A
E A F
```

```
JuegoSweetCrush - Debug [X]

5. Agregar fila
6. Eliminar fila
7. Agregar columna
8. Eliminar columna
9. Mostrar estadísticas
0. Salir
Seleccione una opcion: 9

=====
ESTADISTICAS
=====

Filas actuales: 5
Columnas actuales: 3
Bytes asignados: 6
Eliminaciones del usuario: 2
Fichas eliminadas: 6
Combinaciones detectadas: 2
Cascadas: 2
Puntaje: 60
```

Aquí podemos ver de nuevo que las estadísticas se van actualizando

Eliminar fila.

```
JuegoSweetCrush - Debug
=====
MENU
=====
1. Mostrar tablero
2. Realizar jugada
3. Mostrar bits
4. Mostrar Fichas en binario
5. Agregar fila
6. Eliminar fila
7. Agregar columna
8. Eliminar columna
9. Mostrar estadísticas
0. Salir
Seleccione una opcion: 6

Ingrese la fila que desea eliminar: 1
Fila eliminada correctamente.
```

```
=====
MENU
=====
1. Mostrar tablero
2. Realizar jugada
3. Mostrar bits
4. Mostrar Fichas en binario
5. Agregar fila
6. Eliminar fila
7. Agregar columna
8. Eliminar columna
9. Mostrar estadísticas
0. Salir
Seleccione una opcion: 1

Tablero: ya no está
A B C
D R C
B E A
E A F
```

Agregar columna.

```
=====
MENU
=====
1. Mostrar tablero
2. Realizar jugada
3. Mostrar bits
4. Mostrar Fichas en binario
5. Agregar fila
6. Eliminar fila
7. Agregar columna
8. Eliminar columna
9. Mostrar estadísticas
0. Salir
Seleccione una opcion: 7

Ingrese la posicion de la nueva columna: 1
Columna agregada correctamente.
```

```
JuegoSweetCrush - Debug
=====
MENU
=====
1. Mostrar tablero
2. Realizar jugada
3. Mostrar bits
4. Mostrar Fichas en binario
5. Agregar fila
6. Eliminar fila
7. Agregar columna
8. Eliminar columna
9. Mostrar estadísticas
0. Salir
Seleccione una opcion: 1

Tablero:
A B B C
D F R C
B F E A
E E A F
```

Eliminar columna.

```
JuegoSweetCrush - Debug x
=====
MENU
=====
1. Mostrar tablero
2. Realizar jugada
3. Mostrar bits
4. Mostrar Fichas en binario
5. Agregar fila
6. Eliminar fila
7. Agregar columna
8. Eliminar columna
9. Mostrar estadísticas
0. Salir
Seleccione una opcion: 8

Ingrese la columna que desea eliminar: 2
Columna eliminada correctamente.
```

```
JuegoSweetCrush - Debug x
MENU
=====
1. Mostrar tablero
2. Realizar jugada
3. Mostrar bits
4. Mostrar Fichas en binario
5. Agregar fila
6. Eliminar fila
7. Agregar columna
8. Eliminar columna
9. Mostrar estadísticas
0. Salir
Seleccione una opcion: 1

Tablero:
A B C
D F C
B F A
E E F
```

→ ya no está

con esto también verificamos que se eliminaron y agregaron columnas correctamente.

finalizar juego .

```
=====
MENU
=====
1. Mostrar tablero
2. Realizar jugada
3. Mostrar bits
4. Mostrar Fichas en binario
5. Agregar fila
6. Eliminar fila
7. Agregar columna
8. Eliminar columna
9. Mostrar estadísticas
0. Salir
Seleccione una opcion: 0

Programa finalizado.
=====
Luna and Oscar
=====

Process exited with code: 0.█
```

Conclusiones.

A través del desarrollo del proyecto, logramos aplicar diversos conceptos de C++, como el manejo de punteros, memoria dinámica, arreglos, funciones y operaciones con bits. El uso de 3 bits por ficha permitió una representación más compacta del tablero. Aprendimos que trabajar directamente con bits exige atención a la posición de cada dato, especialmente cuando una ficha se encuentra dividida entre dos bytes.

La organización del programa en archivos separados: `main.cpp`, `tablero.cpp`, `tablero.h`, `juego.cpp` y `juego.h`, facilitó la gestión de las distintas partes del proyecto. Finalmente, esta experiencia nos permitió vincular la representación de información a nivel de bits con la lógica del juego, abarcando aspectos como la eliminación de fichas, la gravedad, la generación de nuevas fichas, combinaciones, cascadas y cambios en las dimensiones del tablero.